Basic arithmetic

- Addition and subtraction of binary and hexadecimal numbers follow the same “carry” and “borrow” rules of decimal arithmetic

- A few examples of hexadecimal addition and subtraction follow

Hexadecimal addition

- Example 1: **2B + 53**

Step 1. **B** + **3** is, in decimal, 11 + 3 = 14 (hexadecimal **E**)
Step 2. **2** + **5** = **7**

```
  2B
+53
----
  7E
```

The result, stored as binary in the computer memory: **01111110** (**0111** is **7**, **1110** is **E**).

Hexadecimal addition

- Example 2: **2B + 4A**

Step 1. **B** + **A** is, in decimal, 11 + 10 = 21 (hexadecimal **15**)
Step 2. From hexadecimal **15**, we take **5** and carry **1**
Step 3. **2** + **4** + **1** (carry) = **7**

2B
+4A
75

The result, stored as binary in the computer memory: **01110101** (**0111** is **7**, **0101** is **5**).

Hexadecimal addition

- Example 3: adding the same numbers in decimal, hexadecimal and binary

Decimal	**Hexadecimal**	**Binary**
6773	1A75	1101001110101
+7772	+1E5C	+1111001011100
14545	38D1	11100011010001

- For large binary numbers, it is more practical (because it takes fewer steps) to convert to hexadecimal, carry out arithmetic operations, then convert the result back to binary

Hexadecimal subtraction

- Example 4: **7A - 2B**

Step 1. **A** < **B**, so borrow hexadecimal **10**

Step 2. **1A** - **B** is, in decimal, 26 - 11 = 15 (hexadecimal **F**)

Step 3. **7** - **2** - **1** (borrowed) = **4**

7A
-2B
4F

The result, stored as binary in the computer memory: **01001111** (**0100** is **4**, **1111** is **F**).

Hexadecimal subtraction

- Example 5: **2B - 7A**

Step 1. **2B** < **7A**, so calculate **- (7A - 2B)**

Step 2. From the previous example, the result is **-4F**

2B
-7A

-4F

The result, stored as binary in the computer memory: *wait a minute…*

…how does the z/Architecture represent negative binary numbers?

- This is presented next

Binary integers

- z/Architecture computers represent binary integers using **fixed-length** fields:
  - 16 bits, called **halfwords** (for example, 005B)
  - 32 bits, called **fullwords** (for example, 0000005B)
  - 64 bits, called **doublewords** (for example, 0000000000005B)
- Binary integers can be *signed* or *unsigned* (also called *logical*)
- Signed integers can be positive or negative
- Unsigned and positive integers are represented as you would expect
- Negative numbers are represented using *two's complement notation* (coming soon)

Unsigned binary integers

- An **unsigned halfword** integer can represent 65,536 different numbers, from 0 to 65,535 (2^{16} - 1):

Binary	Hexadecimal	Decimal
0000 0000 0000 0000	0000	0
0000 0000 0000 0001	0001	1
…	…	…
1111 1111 1111 1110	FFFE	65,534
1111 1111 1111 1111	FFFF	65,535

Unsigned binary integers

- An **unsigned fullword** integer can represent numbers from 0 to 4,294,967,295 (2^{32} - 1):

Binary	Hexadecimal	Decimal
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000	0
0000 0000 0000 0000 0000 0000 0000 0001	0000 0001	1
…	…	…
1111 1111 1111 1111 1111 1111 1111 1110	FFFF FFFE	4,294,967,294
1111 1111 1111 1111 1111 1111 1111 1111	FFFF FFFF	4,294,967,295

Unsigned binary integers

- An **unsigned doubleword** integer can represent numbers from 0 to 18,446,744,073,709,551,615 (2^{64} - 1):

Binary	Hexadecimal	Decimal
Too long!	0000 0000 0000 0000	0
	0000 0000 0000 0001	1
	…	…
	FFFF FFFF FFFF FFFE	18,446,744,073,709,551,614
	FFFF FFFF FFFF FFFF	18,446,744,073,709,551,615

Signed binary integers

- In signed integers, *the leftmost bit (bit 0) is the sign:* **0 for positive**, **1 for negative**. The remaining bits represent the number
- **Positive** binary integers are in true binary notation with **sign bit 0**
- **Negative** binary integers are in *two's-complement notation,* with **sign bit 1**

Negative binary integers

- To form a two's complement negative integer:

    1. Invert each bit of the positive integer
    2. Add 1

- The same process converts negative numbers to positive

- Next, an example

Negative binary integers

Example 1: represent decimal **-95** in a binary halfword

	Decimal	Hexadecimal	Binary
Positive value	95	005F	0000 0000 0101 1111
Invert bits		FFA0	1111 1111 1010 0000
Add 1			+1
Negative number	-95	FFA1	**1111 1111 1010 0001**

To represent -95 in a fullword or doubleword, **extend the sign bit to the left**:

1111 1111 .... **1**111 1111 1010 0001

Negative binary integers

Example 2: convert the binary halfword representation of **-95** back to **95**

	Decimal	Hexadecimal	Binary
Negative value	-95	FFA1	1111 1111 1010 0001
Invert bits		005E	0000 0000 0101 1110
Add 1			+1
Positive number	95	005F	**0000 0000 0101 1111**

To represent 95 in a fullword or doubleword, **extend the sign bit to the left**:

0000 0000 .... **0**000 0000 0101 1111

Negative binary integers

Example 3: represent decimal **-1** in a binary halfword

	Decimal	Hexadecimal	Binary
Positive value	1	0001	0000 0000 0000 0001
Invert bits		FFFE	1111 1111 1111 1110
Add 1			+1
Negative number	-1	FFFF	**1111 1111 1111 1111**

To represent -1 in a fullword or doubleword, **extend the sign bit to the left**:

1111 1111 .... **1**111 1111 1111 1111

Signed binary integers

- A **signed halfword** integer can represent numbers from -32,768 (-2^{15}) to +32,767 ($2^{15} - 1$):

Binary	Hexadecimal	Decimal
1000 0000 0000 0000	8000	-32,768
1000 0000 0000 0001	8001	-32,767
1111 1111 1111 1111	FFFF	-1
0000 0000 0000 0000	0000	0
0000 0000 0000 0001	0001	1
0111 1111 1111 1110	7FFE	32,766
0111 1111 1111 1111	7FFF	32,767

Signed binary integers

- A **signed fullword** integer can represent numbers from -2,147,483,648 (-2^{31}) to +2,147,483,647 ($2^{31} - 1$):

Binary	Hexadecimal	Decimal
1000 0000 0000 0000 0000 0000 0000 0000	8000 0000	-2,147,483,648
1000 0000 0000 0000 0000 0000 0000 0001	8000 0001	-2,147,483,647
1111 1111 1111 1111 1111 1111 1111 1111	FFFF FFFF	-1
0000 0000 0000 0000 0000 0000 0000 0000	0000 0000	0
0000 0000 0000 0000 0000 0000 0000 0001	0000 0001	1
0111 1111 1111 1111 1111 1111 1111 1110	7FFF FFFE	2,147,483,646
0111 1111 1111 1111 1111 1111 1111 1111	7FFF FFFF	2,147,483,647

Signed binary integers

- A **signed doubleword** integer can represent numbers from -9,223,372,036,854,775,808 (-2^{63}) to +9,223,372,036,854,775,807 ($2^{63} - 1$):

Hexadecimal	Decimal
8000 0000 0000 0000	-9,223,372,036,854,775,808
8000 0000 0000 0001	-9,223,372,036,854,775,807
FFFF FFFF FFFF FFFF	-1
0000 0000 0000 0000	0
0000 0000 0000 0001	1
7FFF FFFF FFFF FFFE	9,223,372,036,854,775,806
7FFF FFFF FFFF FFFF	9,223,372,036,854,775,807

Unit summary

This concludes the introduction to numbering systems. Hope you liked it!

Having completed this unit, you should be able to:

Unit summary

This concludes the introduction to numbering systems. Hope you liked it!

Having completed this unit, you should be able to:

- Describe a binary numbering system (base 2), a decimal numbering system (base 10) and a hexadecimal numbering system (base 16), and be able to convert values between them

Unit summary

This concludes the introduction to numbering systems. Hope you liked it!

Having completed this unit, you should be able to:

- Describe a binary numbering system (base 2), a decimal numbering system (base 10) and a hexadecimal numbering system (base 16), and be able to convert values between them
- Complete basic arithmetic operations (add, subtract) in any of the three numbering systems

Unit summary

This concludes the introduction to numbering systems. Hope you liked it!

Having completed this unit, you should be able to:

- Describe a binary numbering system (base 2), a decimal numbering system (base 10) and a hexadecimal numbering system (base 16), and be able to convert values between them
- Complete basic arithmetic operations (add, subtract) in any of these numbering systems
- Explain z/Architecture’s signed binary number representation and “two’s complement notation”